

1: Introduction to Mobile Computing & App Ecosystem
ความสำคัญและนิยามของ Mobile Computing
ทำไมต้องเรียนเรื่องนี้? อุปกรณ์มือถือกลายเป็นอุปกรณ์หลักที่คนเข้าถึงอินเทอร์เน็ตมากที่สุด (แห่งหนึ่ง Desktop) และแอปพลิเคชันเป็นหัวใจหลักในการขับเคลื่อนอุตสาหกรรมระดับโลก (FinTech, Social, Health, Commerce)
นิยามของ Mobile Computing: คือการประมวลผลทางคอมพิวเตอร์ที่ทำผ่าน "อุปกรณ์พกพา" ที่ "เชื่อมต่อเครือข่ายได้" (เช่น สมาร์ทโฟน, แท็บเล็ต, สมาร์ทวอตช์)
ลักษณะเฉพาะตัว (Characteristics): มีความคล่องตัวสูง (Mobility), อยู่กับเราทุกที่ทุกเวลา (Ubiquity), และมีความสามารถในการรับรู้สภาพแวดล้อมผ่านเซนเซอร์ (Context-awareness)
ความแตกต่างระหว่าง Mobile กับ Desktop (ข้อจำกัดที่ต้องระวังในการพัฒนาแอป)
ทรัพยากรมีจำกัด (Limited Resources): ต้องบริหารจัดการแบตเตอรี่, พลังการประมวลผล (CPU), และพื้นที่หน่วยความจำให้ดีกว่าการทำโปรแกรมบน PC
เครือข่ายแปรปรวน (Variable Network): สภาพแวดล้อมอินเทอร์เน็ตไม่เสถียรเสมอไป
หน้าจอและ UI: หน้าจอมีขนาดเล็ก และต้องออกแบบโดยคำนึงถึง "การสัมผัส" เป็นหลัก (Touch-first UI)
นโยบายและสิทธิ์ (OS Policies): ระบบมือถือจะมีการจัดการที่เข้มงวด เช่น การทำ Sandboxing และระบบ Permission
ความแตกแยกของฮาร์ดแวร์ (Device Fragmentation): ต้องรองรับหลายขนาดหน้าจอและสเปกที่แตกต่างกัน
ภาพรวมระบบนิเวศ (Mobile App Ecosystem) และ OS
องค์ประกอบหลัก: OS vendors, App Stores, นักพัฒนา, ฮาร์ดแวร์, และเครื่องมือ (SDK, Cloud, CI/CD)
Android OS: Open-source (AOSP), ใช้ Java/Kotlin, ตลาดใหญ่ แต่ Fragmentation สูง
iOS OS: Closed-source, ใช้ Swift/Objective-C, Optimization ดี แต่กฎ App Store เข้มงวด
เจาะลึกประเภทของ Mobile App (Type of Applications)
Native Apps: Performance สูงสุด เข้าถึง API เต็ม แต่ต้องเขียนแยก
Cross-Platform Apps: โค้ดเดียวรันหลายระบบ (Flutter, React Native) พัฒนาเร็ว
Hybrid Apps: ใช้ Web + Native wrapper เร็วแต่ performance ต่ำกว่า
Web Apps / PWA: ไม่ต้องติดตั้ง พัฒนาเร็วสุด แต่เข้าถึง hardware ได้น้อย
เทรนด์การพัฒนาและบทบาทของ Framework
Major Trends: Declarative UI (Flutter, SwiftUI), AI ช่วยเขียนโค้ด, Multi-device, 5G + Edge computing
Framework มีไว้เพื่อ: ลดความซ้ำซ้อนของ OS, จัดการ UI/Lifecycle, เพิ่ม Productivity
Flutter 101 (พื้นฐาน Dart & Flutter)
เหตุผลที่เรียน Flutter: โค้ดสั้น, พัฒนาเร็ว, Cross-platform, รองรับ Material Design
โครงสร้างหลัก: main() คือ entry point
runApp(): เริ่ม Framework และแสดง UI
หัวใจ: "Everything is a Widget" ทุกอย่างคือ Widget และเป็น Class ที่ override build() ได้
// จุดเริ่มต้นการรันแอป
void main() => runApp(const MyApp());
// โครงสร้างคลาสของ Widget
class PaddedText extends StatelessWidget {
const PaddedText({super.key});
@override
Widget build(BuildContext context) {
return Padding(
padding: const EdgeInsets.all(8.0),
child: const Text("Hello, World!"),);}

2: Characteristics and Constraints of Mobile App
1. ลักษณะเฉพาะของ Mobile Computing (ทำให้ออกต่างจาก Desktop?)
• Ubiquity (ความมีอยู่ทุกหนทุกแห่ง): อุปกรณ์มือถือติดตัวผู้ไปทุกบริบท (บ้าน, ที่ทำงาน, กำลังเดินทาง) ผู้ใช้จึงคาดหวังให้แอปตอบสนองได้ "ทันที" (Instant response) มีความพร้อมใช้งานตลอดเวลา และข้อมูลจึงต้องรั้งกันทุกอุปกรณ์
• Micro-usage Pattern: ผู้ใช้มักจะหยิบมือถือขึ้นมาใช้ในระยะเวลาย่าง (เช่น กดดูแจ้งเตือน 10 วินาที) ทำให้การออกแบบ UI ต้องกะทัดรัด เข้าถึงง่าย
• Context-Awareness (การรับรู้บริบท): แอปมือถือต้องปรับตัวตามสภาพแวดล้อมได้แบบไดนามิก - Dynamic context: ผู้ใช้กำลังเดิน, วิ่ง, ขับรถ หรืออยู่ใต้อาคาร - Device context: แบตเตอรี่เหลือน้อย, สัญญาณเน็ตหาย - Environment context: แสงสว่างจ้า, เสียงดังรอบข้าง
2. กรณีศึกษา (Case Studies) ในการรับมือกับข้อจำกัด
• Twitter Case (2012): แสดงให้เห็นถึง "ข้อจำกัดด้านสถาปัตยกรรมข้อมูล" เมื่อระบบต้องรับมือกับ Requests มหาศาลในเสี้ยววินาที (12k request/sec โดเมนหลัก, 300k request/sec คอนายอ) การเลือกระบบฐานข้อมูลและ Cache จึงชี้เป็นชี้ตายแอป
• Spotify Case (Context-Awareness): ตัวอย่างการรับมือ "เครือข่ายไม่เสถียร" และจะปรับลดความละเอียดวิดีโอ/เสียงอัตโนมัติเมื่อเน็ตช้า (Slow networks) และจะแอบโหลดข้อมูลล่วงหน้า (Preload) เมื่อจับได้ว่าผู้ใช้ต่อ WiFi เพื่อประหยัด Data อินเทอร์เน็ต
3. ข้อจำกัดของอุปกรณ์และระบบ (Device & OS Limitations)
• Hardware: ความเร็ว CPU/GPU และพื้นที่ความจุ (Storage) มีจำกัด
• OS Restrictions: ระบบปฏิบัติการจะจำกัดการทำงานเบื้องหลัง (Background execution) เพื่อประหยัดแบตเตอรี่ มีระบบ Permission ที่เข้มงวด และสามารถ Kill (สั่งปิด) แอปของคุณที่แอบทำงานอยู่เบื้องหลังได้ตลอดเวลาหากระบบต้องการ RAM ไปใช้กับแอปอื่น
• File System (Android): - System Partitions (Read-only): ใ้กับ OS และไดรเวอร์ - App Private Storage (Sandboxed): สำคัญมาก เป็นพื้นที่ปิดตัวของแต่ละแอป ใช้เก็บ Cache, Database, และ Preferences (เอาไว้ข้ามมาขโมยดูไม่ได้) - Shared Storage (Scoped): พื้นที่สาธารณะ เช่น รูปภาพ ต้องขอสิทธิ์ก่อนเข้าถึง
4. กลยุทธ์การทำ Cache (Caching Strategies)
• แนวคิดหลัก: Cache คือการแลกเปลี่ยน (Trade-off) ระหว่าง ความเร็ว (Speed) vs ความสดใหม่ของข้อมูล (Freshness) vs ความซับซ้อนของระบบ (Complexity)
• กฎการ Cache: ควร Cache เฉพาะการทำงานที่หนัก, โหลดช้าบ่อยๆ, หรือข้อมูลที่จำเป็นตอนออฟไลน์ ห้าม Cache ข้อมูลที่เป็น Real-time หรือข้อมูล Sensitive
• ตัวอย่างการวิเคราะห์ในท้องถอบ: - แอปหุ้น (Stock App): ข้อมูลพอร์ตโฟลิโอหรือราคาหุ้น ไม่ควร Cache เพราะต้องอัปเดตวินาทีต่อวินาที - แชตแอป (Chat App): ข้อความที่เพิ่งส่งต่อสัญญาณหาย ควร Cache ไว้ในเครื่องก่อน พอมีสัญญาณค่อย Push ขึ้นเซิร์ฟเวอร์ - แอปช้อปปิ้ง (Shopping App): ชื่อสินค้า, รูปภาพ ควร Cache แต่ สถานะสินค้า (มีของ/หมดสต็อก) ไม่ควร Cache เพราะอาจเกิดปัญหาลูกค้ากดซื้อของที่หมดไปแล้ว
5. ปัญหาด้าน UI/UX และเซนเซอร์ (UI/UX Constraints & Sensors)
• Fat-finger Problem: นมือผู้ใช้ใหญ่กว่าปุ่มบนหน้าจอ ทำให้กดผิดบ่อย (Error-prone input) ต้องออกแบบปุ่มให้ใหญ่และมี Spacing พอเหมาะ
• System UI Intrusions: ขอบปากกล้อง (Notch) หรือ Status bar บังเนื้อหาแอป (ใน Flutter แก้ได้ด้วยการใช้ SafeArea ครอบ Widget)
• Sensor Warning: การดึงข้อมูลจากเซนเซอร์ (GPS, Gyroscope) แบบต่อเนื่อง (Continuous data streams) จะทำให้แบตเตอรี่ไหลเป็นน้ำ (Battery drain) และต้องเช็ก Permission ทุกครั้งก่อนเรียกใช้
6. Dart 101: คำแปรและ Null Safety (หัวใจของภาษา Dart)
• String Interpolation: ใช้ \$ ตัวแปร หรือ \${การคำนวณ} แทรกใน String ได้เลย เช่น print(Value doubled is \$val * 2);
• Null Safety: โดยปกติ Dart ไม่อนุญาตให้ตัวแปรมีค่าเป็น Null (เช่น int a = null; จะ Error ทันที) - วิธีแก้คือเติม ? เข้าไปหลังชนิดตัวแปร เช่น int? a = null; - ตัวดำเนินการเช็ก Null (Null Operators): - b ??= val; หมายถึง ถ้า b ยังเป็น null ให้กำหนดค่า val ลงไป - a = value ?? 0; หมายถึง ถ้า value เป็น null ให้ใช้เลข 0 แทน - myObject?.someProperty หมายถึง ถ้า myObject ไม่ใช่ null ค่อยดึง property ออกมา (ป้องกันแอปพัง)
• Late Assignment (late): ใช้เวลาประกาศตัวแปรใน Class แต่ยังไม่มีความจำเป็นต้องนิยาม ณ ตอนนั้น เป็นการสัญญากับ Compiler ว่า "ห้ามเป็น Null ณ เวลานั้นจะใส่ค่าให้ก่อนที่มันจะถูกเรียกใช้แน่นอน"
(Flow Analysis)
bool isEmptyList(Object object) {
// Dart จะฉลาดพอรู้ว่าผ่านบรรทัดนี้ไปแล้ว object จะกลายเป็น Listแน่นอน
if (object is List) return false;
return object.isEmpty; // ใช้งาน properties ของ List ได้เลยโดยไม่ Error

3: Evolution of Mobile Application Framework
1. สาเหตุที่ Mobile Framework ต้องวิวัฒนาการ (Why Frameworks Evolve?)
• Mobile Framework มีการพัฒนาอย่างรวดเร็วเพื่อตอบสนองต่อการเติบโตของฮาร์ดแวร์, การออกแบบ OS, และความคาดหวังของผู้ใช้งาน
• สาเหตุหลักของการวิวัฒนาการประกอบด้วย: - การเติบโตอย่างรวดเร็วของความสามารถฮาร์ดแวร์มือถือ - ผู้ใช้คาดหวังประสิทธิภาพ, คุณภาพ UI, และการตอบสนองที่รวดเร็ว (Responsiveness) - ความต้องการเครื่องมือที่ช่วยเพิ่มความเร็วในการทำงานของนักพัฒนา (Developer productivity) - ข้อจำกัดของระบบยุคแรก เช่น ปัญหา Fragmentation (ความหลากหลายของฮาร์ดแวร์), เครื่องมือพัฒนาที่ขี้อ่อนแอ, และ API ที่ไม่คงเส้นคงวา - เพราะการสร้างและรองรับหลายแพลตฟอร์ม (Multi-platform) และเชื่อมต่อข้ามอุปกรณ์อย่างราบรื่น
2. ยุคก่อนสมาร์ทโฟน (Pre-Smartphone Era)
• แอปในยุคนี้เน้นไปที่เครื่องมือพื้นฐาน เช่น SMS, เกมง่ายๆ, และการจัดการอุปกรณ์
• การพัฒนาแอปลำบากเพราะฮาร์ดแวร์ไม่แตกต่างกัน และมีข้อจำกัดด้านทรัพยากร (CPU ต่ำ, หน่วยความจำน้อย, และไม่มียูนิคเซนเซอร์รองรับ)
• J2ME (Java ME): - เป็นนิยามมากในยุค Feature phones - มี API ที่พื้นฐานสำหรับสร้าง UI, เครือข่าย, และเก็บข้อมูล - ปัญหาหลักคือ Fragmentation อย่างรุนแรง (ต้องเขียนโค้ดแยกตามขนาดหน้าจอ), UI ไม่สวยงาม และต้องจัดการระบบด้วยตัวเอง (Manual Management)
• Symbian OS (C++): - เป็น OS สมาร์ทโฟนที่ครองตลาดก่อนการมาของ iOS และ Android - รองรับแอปแบบ Native ที่ถือว่า J2ME แต่มีสภาพแวดล้อมการพัฒนาที่ซับซ้อน (Steep learning curve) โค้ดเขียนยืดยาว (Verbose) และต้องกินค่าหน่วยความจำเอง (Manual Memory Management)
• Windows Mobile: - ระบบของ Microsoft ที่สร้างบน Windows CE kernel - เจาะกลุ่มผู้ใช้ระดับองค์กร (Enterprise) เช่น อีเมล, ปฏิทิน, เครื่องมือ Office - รองรับหน้าจอสัมผัสแบบ Resistive และใช้ Stylus ชีบ (ไม่เหมาะกับการใช้นิ้ว) - เขียนด้วย C++ หรือ C# (ผ่าน .NET) เครื่องมือพัฒนาคือ Visual Studio
3. การเติบโตของสมาร์ตโฟน (Rise of Modern Smartphone Frameworks)
• ยุคใหม่เริ่มขึ้นเมื่อ Apple เปิดตัว iPhone (2007) และ Google เปิดตัว Android (2008) นำมาซึ่งระบบที่เน้น "การสัมผัส" (Multi-touch) และการมีเซนเซอร์
• นวัตกรรมสำคัญ: มี App Store / Google Play, มี API มาตรฐานสำหรับกล้องและเซนเซอร์
• iOS (Unified Approach): เป็นระบบที่ถืออุปกรณ์มีน้ำหนักมาตรฐาน รวม SDK และ API ไว้ด้วยกันอย่างเข้มงวด มีเครื่องมือคุณภาพสูงอย่าง Xcode เดิมใช้ภาษา Objective-C และเริ่มใช้ Swift ในปี 2014 โดยมี UIKit เป็นตัวจัดการ UI
• Android (Open Ecosystem): เป็นระบบเปิด (AOSP - Open-source platform) ผู้ผลิตมือถือสามารถนำไปปรับแต่งฮาร์ดแวร์และ UI ได้อิสระ (ก่อให้เกิดความยืดหยุ่นหลากหลาย) ใช้ Linux kernel ผสมกับการเขียนแอปด้วย Java/Kotlin และ XML
4. คลื่นลูกใหม่ของเทคโนโลยี Cross-Platform
• First Wave (Web-Based): ยุคแรกใช้ HTML, CSS, JavaScript รันผ่าน WebView (เช่น PhoneGap) ข้อดีคือทำเสร็จเร็ว (Shorter time-to-market) และต้นทุนต่ำ แต่ประสิทธิภาพและการเข้าถึงฮาร์ดแวร์มีจำกัด
• Second Wave (Native UI): เกิดขึ้นเพราะยุคแรกประสิทธิภาพไม่ดี ยุคนี้จึงเน้นการเขียนโค้ดข้ามแพลตฟอร์มที่เรียกใช้ UI ดั้งเดิมของระบบ (Native feel) - React Native (2015): โดย Facebook ใช้ JavaScript เรียก Native UI Components - Xamarin: ใช้ C# และ .NET ทำงานร่วมกับ Visual Studio
• Modern Frameworks (ยุคปัจจุบัน):
Flutter (2018): โดย Google ให้ความสำคัญ มีจุดเด่นคือ "Built-in Skia rendering engine" (วาดทิกเชลทุกจุดบนจอด้วยตัวเองไม่พึ่งโปรแกรม) ใช้โครงสร้าง Declarative UI และมีฟิวเจอร์ Hot Reload Kotlin Multiplatform (KMP). โดย JetBrains ใช้แนวคิดลอจิกทางธุรกิจ (Business logic) ข้ามแพลตฟอร์ม แต่ส่วน UI จะเขียนแยกเป็น Native (Jetpack Compose หรือ SwiftUI)
5. กรณีศึกษา: วิวัฒนาการของแอป Instagram (ใช้วิเคราะห์ข้อเขียน)
• แอปใหญ่ๆ มักใช้ระบบลูกผสม (Mix of native + cross-platform)
• 2010 - 2014 (เริ่มต้นด้วย Pure Native): ใช้ Objective-C และ Java ส่วนๆ เพราะต้องการประสิทธิภาพสูงสุดในการแสดงผลรูปภาพ, ความลื่นไหลของ Feed, การแต่งฟิลเตอร์กล้อง, และรองรับ Gesture ที่ซับซ้อน
• 2015 - 2017 (เปลี่ยนมาใช้ React Native): นำ RN มาใช้เพื่อให้อยู่ที่เจอรียได้เร็วขึ้น (Faster iteration) และประหยัดต้นทุนระบบ โดยทดลองเริ่มจากหน้าจอที่ "ไม่ต้องการประสิทธิภาพสูง"
• 2017 - 2021 (ระบบผสมผสาน Pragmatic Approach): ใช้ Native สำหรับฟีเจอร์ที่กินสเปคหนัก (เช่น Feed, กล้อง, Reels, แชท) เพื่อ UX ที่ดีที่สุด และใช้ React Native สำหรับหน้าย่อย
• 2021 - ปัจจุบัน (เน้น Native ส่วนๆ): ปัจจุบันทุกชุดขยายการใช้ React Native และกำลังย้ายไปสู่ "Native Declarative Architectures" (SwiftUI สำหรับ iOS และ Jetpack Compose สำหรับ Android) สาเหตุที่ไม่ใช้ Flutter เพราะบริษัทลงทุนกับ Native ไปหมดแล้ว จึงไม่คุ้มที่จะรี้อทำใหม่หมด
6. Flutter 101 Comer (พื้นฐานโค้ดและโครงสร้างสำหรับ Lab)
• Project Folders (โครงสร้างไฟล์เดอร์): /lib: ฟิลเดอร์สำหรับเขียนโค้ดหลักของแอป (main.dart อยู่ที่นี่) /ios, /android, /web: ฟิลเดอร์สำหรับตั้งค่าเฉพาะระบบปฏิบัติการนั้นๆ /test: ฟิลเดอร์สำหรับเขียนเทสต์
• pubspec.yaml: เป็นไฟล์คอนฟิกโปรเจกต์ ใช้จัดการเพิกง (Dependencies) และต้องเปิดใช้งาน uses-material-design: true เพื่อใช้ไอคอนและดีไซน์แบบ Material
• Everything is a Widget: ทุกสิ่งทีแสดงผลบนหน้าจอ Flutter คือ Widget - Stateless Widget: UI ที่อาศัยแค่ข้อมูลเริ่มต้นที่ส่งเข้ามา (Configuration information) ไม่มีการเปลี่ยนแปลงสถานะตัวเอง - Stateful Widget: UI ที่สามารถเปลี่ยนแปลงตัวเองแบบไดนามิกได้จากตัวแปรภายใน (Internal clock-driven state) หรือขึ้นอยู่กับสถานะระบบ
• องค์ประกอบ Layout ขึ้นพื้นฐาน (Layout Structures):
• runApp(): คำสั่งเริ่มต้นใน main() เพื่อเริ่มการทำงาน Framework
• MaterialApp: Widget หลักที่เป็นราก (Root) ของแอป จัดเตรียมโครงสร้างสำคัญเช่น Navigator (และนำไปใช้สำหรับแอนิเมชัน)
• Scaffold: โครงสร้างหน้าจอ ประกอบด้วยพารามิเตอร์สำคัญ: - appBar: แถบบาร์แนวอนด้านบน - body: พื้นที่หลักตรงกลางแอป - floatingActionButton: ปุ่มวงกลมลอยตัวที่มุมล่างขวา - bottomNavigationBar: แถบเมนูด้านล่างสุดของแอป

